

# GPCSSI Log Anomaly Detector

A Complete Project Guide

GPCSSI Program — Cybersecurity & AI

**Project:** GPCSSI-FINAL

**Repository:** [github.com/Narwal-himanshu/GPCSSI-FINAL](https://github.com/Narwal-himanshu/GPCSSI-FINAL)

**Tech:** FastAPI • Ollama • Gemini • Python

**Purpose:** LLM-powered Log Anomaly Detection

# Table of Contents

- 1. What Is This Project?
- 2. The Problem It Solves
- 3. Tech Stack (Simple Explanations)
- 4. Project Structure
- 5. Architecture & Pipeline
- 5a. ML Pre-filter (Log-Anomaly-Detector Integration)
- 6. Analysis Modes Explained
- 7. Local vs Cloud: Ollama vs Gemini
- 8. How To Run The Project
- 9. Alternatives & Comparison
- 10. Why This Project Stands Out
- 11. Limitations & Future Work

## 1. What Is This Project?

---

**GPCSSI-FINAL** is a web application that lets you upload server log files (like Apache access logs or Linux system logs) and automatically detects *anomalies* — unusual, suspicious, or error-related lines — using Artificial Intelligence.

Instead of manually scanning through thousands of log lines, you upload the file to a web page, pick an AI model, and get back a clean report showing only the problematic lines with explanations of *why* each one is suspicious.

The app was built as part of the **GPCSSI program** (Google Cybersecurity and Safety Initiative) to demonstrate how Large Language Models (LLMs) can be applied to real-world security monitoring tasks.

**Simple Analogy:** Imagine you have a 500-page server diary. Reading all of it would take hours. This app is like giving that diary to a smart assistant who quickly flips through every page and flags only the suspicious entries, telling you *why* each one caught their eye.

## 2. The Problem It Solves

---

### Why is log analysis hard?

Servers generate thousands of log lines every hour. A typical Apache web server log can have 50,000+ lines in a single day. Manually finding security issues, errors, or unusual patterns in that much text is:

- **Time-consuming:** Hours of reading repetitive text
- **Error-prone:** Humans miss things after the 1000th line
- **Skill-dependent:** You need to know what "normal" looks like for each log format

### Why not use traditional tools?

Traditional tools like **Splunk** or **ELK Stack** rely on:

- **Rules/Regex:** You write patterns like `error|failed|denied`. But what if an anomaly doesn't match any known pattern?
- **Machine Learning models:** These need labeled training data (thousands of "normal" and "anomalous" examples) — which you often don't have.
- **Heavy infrastructure:** Setting up Elasticsearch clusters just to scan a log file is overkill.

### How this project solves it

Instead of rules or trained models, this app uses **Large Language Models (LLMs)** — AI models that understand language and context. The LLM reads each log line (or groups of lines) and decides whether it looks anomalous based on its general understanding of server logs. It doesn't need training, doesn't need regex patterns, and works on any text-based log format.

**Key Insight:** The same LLMs that can write essays and answer questions can also read server logs and tell you if something looks wrong — because they've been trained on vast amounts of text, including technical documentation and code.

### 3. Tech Stack (Simple Explanations)

| Technology               | What It Does                                                   | Why It Was Chosen                                              |
|--------------------------|----------------------------------------------------------------|----------------------------------------------------------------|
| Python 3.9+              | The programming language powering the entire backend           | Widely used, great library support for AI/ML, easy to learn    |
| FastAPI                  | Web framework that handles HTTP requests and serves the API    | Fast (async), automatic docs, simple code, built-in validation |
| Uvicorn                  | ASGI server that runs the FastAPI app                          | Lightning fast, built for async Python, industry standard      |
| Ollama                   | Runs AI models locally on your machine (no internet needed)    | Free, private, works offline, easy to set up                   |
| Google Gemini API        | Cloud-based AI models from Google                              | More powerful models, faster inference, higher accuracy        |
| HTML / CSS / JS          | The frontend web page users interact with                      | Zero dependencies — works in any browser, lightweight          |
| SSE (Server-Sent Events) | Streams progress updates from backend to frontend in real-time | Simpler than WebSockets, perfect for one-way data flow         |
| python-multipart         | Handles file uploads from the browser                          | Required by FastAPI for multipart form data                    |
| NumPy / Pandas           | Data processing libraries                                      | Available for future data analysis features                    |

#### Dependencies (requirements.txt)

```
numpy
pandas
fastapi
uvicorn
python-multipart
ollama
google-genai
```

That's it — just **7 packages**. The entire project has minimal dependencies, making it easy to install and run.

## 4. Project Structure

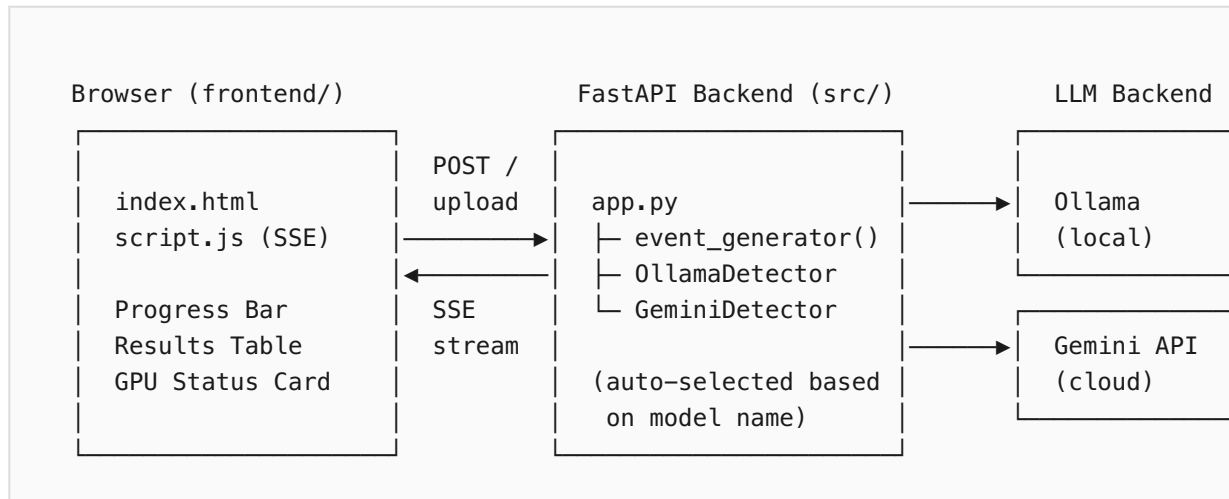
---

```
GPCSSI-FINAL/
├── README.md                # Project documentation
├── requirements.txt         # Python dependencies (7 packages)
├── src/                    # Backend Python source code
│   ├── app.py              # FastAPI entry point (routes, SSE)
│   ├── gemini_detector.py  # Gemini-powered anomaly detector
│   └── ollama_detector.py  # Ollama-powered anomaly detector
├── frontend/              # Web UI (served as static files)
│   ├── index.html          # Main page with upload form
│   ├── script.js           # Frontend logic (API, SSE, GPU)
│   └── style.css           # Visual styling
├── data/                  # Sample log datasets
│   ├── Apache.log          # 56,482 lines of Apache server logs
│   └── Linux.log           # 25,567 lines of Linux syslog
├── examplelog/            # Generated example logs
│   └── my_access.log.txt   # 5,000 synthetic Apache access logs
└── scripts/              # Setup utilities
    ├── genlog.py           # Synthetic log generator (CLI tool)
    ├── setup_ollama.sh     # Ollama setup for macOS/Linux
    └── setup_ollama.ps1    # Ollama setup for Windows
```

| File                                  | What It Does                                                                                                                                                                                       |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>src/app.py</code>               | Main server: defines all API routes ( <code>/upload-log/</code> , <code>/api/gpu-status</code> , <code>/api/validate-api-key</code> ), orchestrates the analysis pipeline, streams results via SSE |
| <code>src/ollama_detector.py</code>   | Contains <code>OllamaDetector</code> class: sends log lines/chunks to local Ollama models, handles retries, parses JSON responses                                                                  |
| <code>src/gemini_detector.py</code>   | Contains <code>GeminiDetector</code> class: sends log lines/chunks to Google Gemini API, implements rate limiting and exponential backoff retries                                                  |
| <code>frontend/index.html</code>      | The web page: file upload form, model/mode/speed selectors, progress bar, results table                                                                                                            |
| <code>frontend/script.js</code>       | Client logic: validates API keys, calls upload endpoint via fetch, reads SSE stream, updates the DOM with results                                                                                  |
| <code>frontend/style.css</code>       | Visual styling: dark/light theme, responsive layout, animated progress bar                                                                                                                         |
| <code>data/Apache.log</code>          | 56,482 lines of real Apache web server logs from 2005 for testing                                                                                                                                  |
| <code>data/Linux.log</code>           | 25,567 lines of real Linux syslog (Red Hat boot sequence) for testing                                                                                                                              |
| <code>scripts/genlog.py</code>        | CLI tool to generate synthetic Apache logs with configurable error rates                                                                                                                           |
| <code>scripts/setup_ollama.sh</code>  | Bash script: installs Ollama, starts service, pulls default models                                                                                                                                 |
| <code>scripts/setup_ollama.ps1</code> | PowerShell equivalent for Windows users with GPU detection                                                                                                                                         |

## 5. Architecture & Pipeline

### High-Level Architecture



### Step-by-Step Pipeline

1. **User loads the web app** at `http://localhost:8000`
2. **On page load**, JavaScript:
  - Calls `GET /api/gpu-status` to check NVIDIA GPU + Ollama health
  - Loads saved Google API key from `localStorage` and auto-validates it
3. **User selects options**: Model (Llama/Qwen/Gemini), Mode (Auto/Line-by-Line/Intensive/Chunking), Speed (Balanced/Fast/Max)
4. **User uploads a log file** and clicks "Analyze Logs"
5. **Frontend sends** `POST /upload-log/` with the file + form parameters
6. **Backend decides which detector to use**:
  - If model starts with `"gemini-"` → **GeminiDetector**
  - Otherwise → **OllamaDetector**
7. **File is parsed**: read, decoded to UTF-8, split into lines (blank lines skipped)
8. **Lines are grouped** based on the selected mode (chunk size determined by mode)
9. **LLM processes chunks concurrently** (concurrency controlled by speed setting via `asyncio.Semaphore`)
10. **Results are streamed** back via SSE as JSON events: progress updates + final anomaly report
11. **Frontend displays**: progress bar, total logs count, anomalies found, and a detailed table



## Key Code: Upload Route (src/app.py:130-147)

```
@app.post("/upload-log/")
async def upload_log(
    file: UploadFile = File(...),
    model: str = Form("llama3.2:1b"),
    mode: str = Form("auto"),
    speed: str = Form("balanced"),
    google_api_key: str = Form(None),
    chunk_percentage: int = Form(10)
):
    if model.startswith("gemini-"):
        speed_map = {"balanced": 1, "fast": 2, "max": 3}
    else:
        speed_map = {"balanced": 2, "fast": 4, "max": 6}
    max_concurrency = speed_map.get(speed, 2)
    return StreamingResponse(
        event_generator(file, model, mode, max_concurrency,
                        google_api_key, chunk_percentage),
        media_type="text/event-stream"
    )
```

## Key Code: Analysis Orchestration (src/app.py:149-216)

```
async def event_generator(file, model, mode, max_concurrency,
                          google_api_key, chunk_percentage):
    contents = await file.read()
    lines = contents.decode("utf-8", errors="replace").splitlines()
    logs_to_analyze = []
    for line_num, line in enumerate(lines, 1):
        if not line.strip(): continue
        logs_to_analyze.append((line_num, line))

    if model.startswith("gemini-"):
        detector = GeminiDetector(model_name=model, api_key=google_api_key,
                                max_concurrency=max_concurrency)
    else:
        detector = OllamaDetector(model_name=model,
                                max_concurrency=max_concurrency)

    # ... async queue, progress callbacks, SSE streaming ...
```

## Key Code: Ollama Chunk Analysis (src/ollama\_detector.py:107-150)

```
async def _analyze_chunked(self, logs, chunk_size=10, progress_callback=None):
    chunks = [logs[i:i + chunk_size]
               for i in range(0, len(logs), chunk_size)]
    semaphore = asyncio.Semaphore(self.max_concurrency)

    async def process_chunk(chunk):
        async with semaphore:
            chunk_text = "\n".join(
                [f"Line {ln}: {c}" for ln, c in chunk])
            prompt = f"""Analyze the following batch of logs for anomalies.
Identify any lines that are unusual, indicate errors, or show suspicious
patterns.
Respond in JSON format as a list of objects, each containing
'line' (integer), 'anomaly' (boolean), and 'reason' (string).
Only include lines that ARE anomalies.
Logs: {chunk_text}"""
            response = await self._call_ollama(prompt)
            batch_results = json.loads(response['response'])
            # ... parse results, filter anomalies ...
```

## Key Code: Gemini Rate Limiter (src/gemini\_detector.py:15-36)

```
class RateLimiter:
    """Token bucket rate limiter to prevent API burst limits."""
    def __init__(self, requests_per_second=0.5, burst=2):
        self.rate = requests_per_second
        self.burst = burst
        self.tokens = burst

    async def acquire(self):
        async with self._lock:
            now = time.monotonic()
            elapsed = now - self.last_refill
            self.tokens = min(self.burst,
                             self.tokens + elapsed * self.rate)
            if self.tokens >= 1:
                self.tokens -= 1
                return
            wait = (1 - self.tokens) / self.rate
            await asyncio.sleep(wait)
```

### 5a. ML Pre-filter Integration (Two-Stage Pipeline)

---

A **pre-filter step** has been added to dramatically reduce the number of lines the LLM needs to analyze. Inspired by [AlCoE/log-anomaly-detector](#), it uses an unsupervised ML approach before the LLM stage.

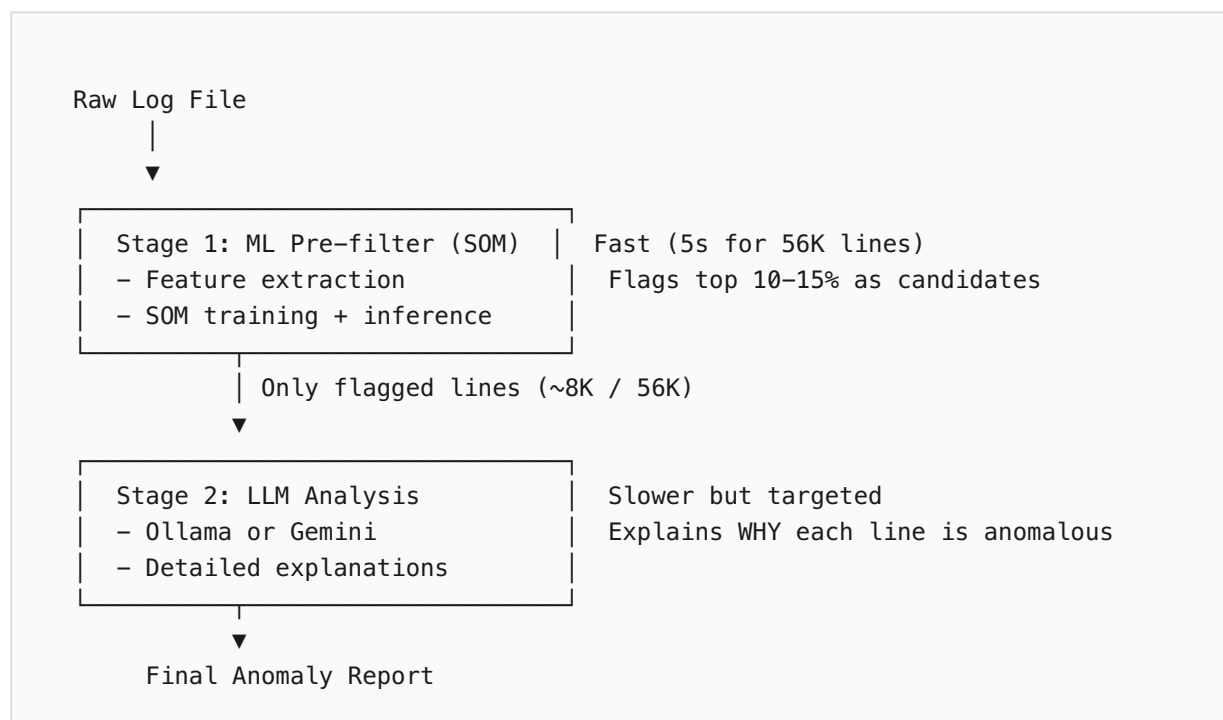
#### Why a pre-filter?

A 50,000-line log file would take the LLM a very long time to process (potentially thousands of API calls). The pre-filter quickly scans all lines and passes only the most "unusual" ones to the LLM, saving time and API costs.

#### How it works

1. **Feature extraction:** Each log line is converted to a 44-dimensional numerical vector (character frequencies, digit ratio, special chars, word stats, etc.)
2. **SOM (Self-Organizing Map):** A neural network grid learns the pattern distribution of all log lines. Each node in the grid represents a "typical pattern."
3. **Scoring:** Each line gets an anomaly score based on how far it is from its closest SOM node, weighted by how rare that SOM node is.
4. **Thresholding:** Lines in the top N% (default: 15%) of scores are flagged as potential anomalies and passed to the LLM.

## Two-Stage Pipeline



## Integration with log-anomaly-detector

The `src/prefilter.py` module implements the core algorithm from **AICoE/log-anomaly-detector** (Word2Vec-style text encoding + Self-Organizing Map) adapted as a lightweight Python class with only `numpy` as a dependency. The original package is archived and has complex dependencies (`gensim`, `sompy`, etc.), so the core logic was reimplemented for seamless integration.

## Key Code: Pre-filter Integration (src/app.py:172-198)

```
# --- ML Pre-filter Step ---
if use_prefilter and len(logs_to_analyze) > 50:
    prefilter = LogPrefilter()
    texts = [content for _, content in logs_to_analyze]

    yield {"type": "prefilter", "status": "training", "total": original_count}

    loop = asyncio.get_event_loop()
    await loop.run_in_executor(None, prefilter.fit, texts)

    results = prefilter.filter(logs_to_analyze)
    filtered = [(line_num, content) for line_num, content, score in results]

    yield {"type": "prefilter", "status": "done",
          "total": original_count, "flagged": len(filtered),
          "filtered_out": original_count - len(filtered)}

    logs_to_analyze = filtered
```

## Key Code: SOM Anomaly Scoring (src/prefilter.py)

```
def _compute_score(self, vector):
    # Distance from line to closest SOM node
    distances = np.sum((self.som - vector) ** 2, axis=2)
    bmu = np.unravel_index(np.argmin(distances), distances.shape)
    min_dist = float(np.min(distances))

    # Penalty for rare BMUs (few lines map here)
    hits = self.bmu_hits[bmu[0], bmu[1]]
    freq_penalty = 1.0 / max(math.sqrt(hits), 0.01)

    # Combined score
    return min_dist * freq_penalty
```

## Performance Comparison

| File Size    | Without Pre-filter        | With Pre-filter (top 15%)   | Speedup |
|--------------|---------------------------|-----------------------------|---------|
| 1,000 lines  | 100 LLM calls (auto mode) | 150 lines → 15 LLM calls    | 6.7x    |
| 10,000 lines | 1,000 LLM calls           | 1,500 lines → 150 LLM calls | 6.7x    |
| 56,000 lines | 5,600 LLM calls           | 8,400 lines → 840 LLM calls | 6.7x    |

**Note:** The pre-filter runs locally in ~5 seconds for 56K lines using only your CPU. The actual LLM calls are what take time, so reducing them by ~85% provides massive speed gains.

## User Interface

The frontend has a toggle switch labeled **"ML Pre-filter"** below the speed selector. When enabled, the UI shows real-time pre-filter progress ("Training on X lines...", "Flagged Y lines out of Z") before the LLM analysis begins. The results also display how many lines were filtered.

## 6. Analysis Modes Explained

The app offers **4 analysis modes** that control how log lines are sent to the LLM:

| Mode                  | How It Works                                                                                                                                   | Best For                                                                         | Speed                      |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|----------------------------|
| <b>Auto (default)</b> | Logs are grouped into chunks of 10 lines each. Each chunk is sent to the LLM at once. The LLM returns a list of anomalies found in that chunk. | General purpose — good balance of speed and accuracy                             | Fast (10x fewer API calls) |
| <b>Line-by-Line</b>   | Each individual log line is sent separately to the LLM. Highest accuracy but slowest.                                                          | Small files where accuracy is critical; NOT available for Gemini (too expensive) | Slow (1 call per line)     |
| <b>Intensive</b>      | Larger chunks of 20 lines each. Sends fewer requests per file.                                                                                 | Very large files where speed is more important than per-line accuracy            | Fastest (fewer calls)      |
| <b>Chunking</b>       | User defines what percentage of total lines each chunk should be. For example, 5% of a 10,000-line file = 500-line chunks.                     | Custom control; fine-tuning for specific file sizes                              | Depends on chunk size      |

### Speed Settings

The **Speed** setting controls how many concurrent requests are sent to the LLM:

| Speed    | Ollama (local) concurrency | Gemini (cloud) concurrency |
|----------|----------------------------|----------------------------|
| Balanced | 2 concurrent requests      | 1 concurrent request       |
| Fast     | 4 concurrent requests      | 2 concurrent requests      |
| Max      | 6 concurrent requests      | 3 concurrent requests      |

**Tip:** Higher concurrency = faster analysis, but may overwhelm weaker hardware (Ollama) or trigger API rate limits (Gemini).

## 7. Local vs Cloud: Ollama vs Gemini

| Feature          | Ollama (Local)                                                   | Gemini (Cloud)                                                  |
|------------------|------------------------------------------------------------------|-----------------------------------------------------------------|
| Privacy          | ✔ Data never leaves your machine                                 | ✗ Data sent to Google servers                                   |
| Cost             | ✔ Completely free                                                | ⚠ Free tier available, then pay-per-token                       |
| Internet needed  | ✔ No — works fully offline                                       | ✗ Yes — requires API access                                     |
| Model size       | Small (0.5B – 1B parameters)<br>Runs on consumer GPUs            | Large (flash: ~60B, pro: ~175B+)<br>Runs on Google's servers    |
| Accuracy         | ⚠ Good for basic anomaly detection, but misses subtle patterns   | ✔ Excellent — much better at understanding context              |
| Speed            | ⚠ Slow on CPU, good on GPU (depends on hardware)                 | ✔ Very fast — Google's infrastructure                           |
| Setup            | Install Ollama + pull models (automated via setup scripts)       | Get a Google API key (free)                                     |
| Rate limits      | ✔ None — local hardware only                                     | ⚠ 429 errors need retry logic (already implemented)             |
| Models available | Llama 3.2 (1B), Qwen 2.5 (0.5B)<br>(any Ollama-compatible model) | Gemini 1.5 Flash, 2.0 Flash<br>(discovered dynamically via API) |

### When to use which?

**Use Ollama when:** Privacy matters, you have no internet, or you want to avoid API costs. Best for learning and experimentation.

**Use Gemini when:** You need the highest accuracy, faster results, or are analyzing large files. Best for production use.



## 8. How To Run The Project

---

### Prerequisites

- Python 3.9+
- pip (Python package installer)
- Ollama installed and running (for local models)
- Google API key (optional, for Gemini models)

### Quick Start

```
# 1. Install Python dependencies
pip install -r requirements.txt

# 2. Set up Ollama (automated script)
# macOS/Linux:
./scripts/setup_ollama.sh
# Windows:
.\scripts\setup_ollama.ps1

# 3. Start the server
uvicorn src.app:app --reload

# 4. Open in browser
open http://localhost:8000
```

### What happens on first load?

The app automatically:

- Detects your NVIDIA GPU via `nvidia-smi`
- Checks if Ollama is running
- Guides you to configure GPU acceleration if needed
- Validates any saved Google API key
- Populates the model dropdown with available options

### GPU Configuration

The app forces Ollama to use CUDA (NVIDIA GPU) instead of Vulkan by setting environment variables at startup:

```
# These are set automatically in src/app.py
os.environ["OLLAMA_VULKAN"] = "0"
os.environ["OLLAMA_LLM_LIBRARY"] = "cuda_v13"
```

## 9. Alternatives & Comparison

---

**How GPCSSI-FINAL compares to existing log analysis tools**

| Tool                                                  | Approach                    | Strengths                                                                    | Weaknesses                                                                                        |
|-------------------------------------------------------|-----------------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <b>GPCSSI-FINAL</b>                                   | LLM-based semantic analysis | Zero setup, works on any log format, explains anomalies, free/offline option | No real-time monitoring, single-user, in-memory only                                              |
| <b>Splunk</b>                                         | Search + rules + ML         | Enterprise-grade, real-time, vast ecosystem, dashboards                      | Very expensive, complex setup, needs query language (SPL), heavy infrastructure                   |
| <b>ELK Stack</b><br>(Elasticsearch, Logstash, Kibana) | Index + search + visualize  | Free (open source), powerful search, real-time ingestion, dashboards         | Heavy (Java-based), requires Elasticsearch cluster, needs grok patterns for parsing, ops overhead |
| <b>Graylog</b>                                        | Search + rules + pipelines  | Free, easier than ELK, built-in pipelines, alerts                            | Still infrastructure-heavy, needs MongoDB + Elasticsearch, less flexible than LLMs                |
| <b>Loki + Promtail</b>                                | Label-based log aggregation | Lightweight (compared to ELK), works with Grafana, cheap storage             | No built-in anomaly detection, limited query capabilities, not for deep analysis                  |
| <b>AI-based (e.g., Datadog Logs, New Relic)</b>       | ML + rules + AI             | Managed service, built-in AI,                                                | Expensive, proprietary, data leaves your network,                                                 |

|  |  |                    |                    |
|--|--|--------------------|--------------------|
|  |  | good<br>dashboards | vendor lock-<br>in |
|--|--|--------------------|--------------------|

## Traditional Tools Flow (e.g., ELK)

```

Log File → Logstash (parse + grok pattern) → Elasticsearch (index) → Kibana (search/visualize)
                                                    ↓
User writes queries/rules
to find anomalies manually

```

## GPCSSI-FINAL Flow

```

Log File → FastAPI (parse lines) → LLM (semantic analysis) → Results (anomalies + reasons)
                        ↑
                No rules, no queries, no training

```

## 10. Why This Project Stands Out

---

**7**

Python packages  
only

**2**

Commands to run

**0**

Training data needed

**2**

LLM providers  
supported

1. **Zero Infrastructure:** Unlike Splunk/ELK that need databases, clusters, and heavy services, this project runs as a single Python process. Install 7 packages and run one command.
2. **No Training, No Rules:** Traditional ML-based anomaly detection needs thousands of labeled examples. This project uses pre-trained LLMs — the model already understands what "normal" and "anomalous" look like from its training data.
3. **Privacy-First Design:** With Ollama support, you can run the entire application offline with no data leaving your computer. None of the alternative tools offer this in a free, easy-to-use package.
4. **Explains Its Reasoning:** The LLM returns natural-language explanations for each anomaly (e.g., "Multiple failed login attempts from same IP in short time"). Traditional tools only show a score or a matched pattern name.
5. **Format-Agnostic:** Works on Apache logs, Linux syslogs, error logs, or any text-based log format — without writing format-specific parsers. Traditional tools require grok patterns or regex for each format.

### What GPCSSI-FINAL is NOT trying to be

- It's **not** a real-time monitoring system (no continuous log tailing)
- It's **not** an enterprise SIEM (no user management, no RBAC)
- It's **not** a replacement for production monitoring tools

It is a **lightweight, educational, developer-friendly tool** for quick ad-hoc log analysis using the power of LLMs. It proves a concept: that modern AI can understand server logs without being explicitly trained for it.

## 11. Limitations & Future Work

---

### Current Limitations

| Limitation              | Explanation                                                                                                                             |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| No authentication       | Anyone with access to the URL can upload files and run analysis. Not suitable for multi-user deployment.                                |
| In-memory only          | Results are not saved. Once you close the page, the analysis is gone. No history, no export.                                            |
| File size limit         | Limited only by server memory. Very large files (>100MB) may crash or slow down significantly.                                          |
| No real-time monitoring | You upload a static file. There's no "tail -f" or continuous log ingestion.                                                             |
| No automated tests      | The repository doesn't include unit or integration tests.                                                                               |
| Small local models      | The bundled Ollama models (1B and 0.5B parameters) are relatively small; they may miss subtle anomalies that larger models would catch. |
| Gemini API costs        | While there's a free tier, heavy usage of Gemini incurs costs based on token count.                                                     |

### Planned Improvements (from README)

- **Custom model support** — particularly Gemma 4 and Gemini 2 Pro for more accurate analysis
- **Better error handling** — more user-friendly messages when models aren't available
- **Configuration file support** — proper .env or config file instead of environment variables

### Potential Future Enhancements

- **Persistent storage** — save analysis results to a database for later review
- **Batch processing** — analyze multiple files at once
- **Export reports** — downloadable PDF/CSV of anomaly reports
- **More LLM providers** — OpenAI, Anthropic Claude, local Hugging Face models
- **Real-time log tailing** — watch a log file and analyze new lines as they appear
- **Docker support** — containerized deployment for easier setup
- **Automated test suite** — CI/CD pipeline with tests

